

Оценка масштабируемости распределенной обработки графов с помощью Galois

Усольцев Данил

Постановка задачи

Цель: исследовать, как меняется время выполнения графовых алгоритмов при росте числа MPI-процессов в Galois Distributed.

Алгоритмы:

- BFS
- PageRank
- SSSP
- Triangle Counting

Основной вопрос: когда распределённое выполнение сокращает полное время запуска, а когда накладные расходы MPI, синхронизации и репликации делают его медленнее?

Экспериментальная методология

Параметры запусков:

- MPI-процессы: 1, 2, 4, 6, 8
- Количество повторов: --runs=20
- Запуск: mpirun --hostfile experiments/hostfile --oversubscribe --bind-to none

Аппаратная и программная среда

- ОС: Ubuntu 24.04.3 LTS
- CPU: 11th Gen Intel Core i5-1135G7 @ 2.40 GHz, 4 ядра, 8 потоков, RAM: 32GB
- Кэш L1: 320 KiB, Кэш L2: 5 MiB, Кэш L3: 8 MiB

Наборы данных

Граф	Вершин	Рёбер	Структура
roadNet-PA	1 090 920	3 083 796	дорожная сеть: вершины связаны почти равномерно, дальние точки соединяются длинными путями
web-Google	916 428	5 105 039	web-граф: есть страницы-хабы с большим числом ссылок, остальные имеют мало связей
wiki-talk-temporal	1 140 149	7 833 140	граф обсуждений: активность распределена неравномерно, есть очень активные участники и темы

Метрики

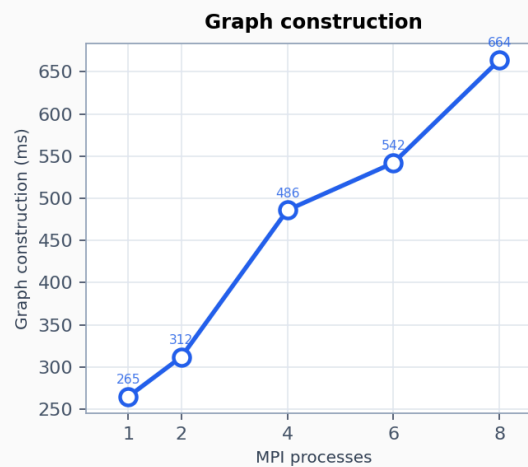
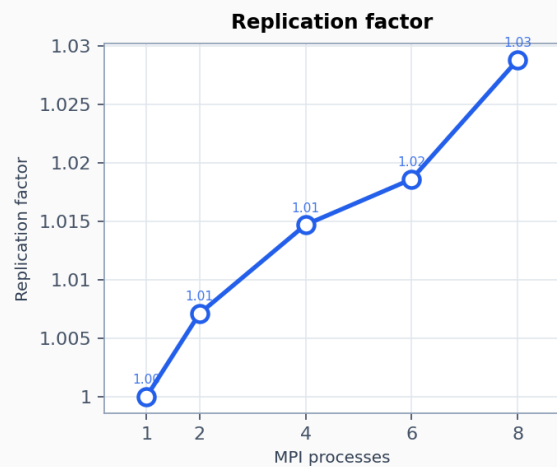
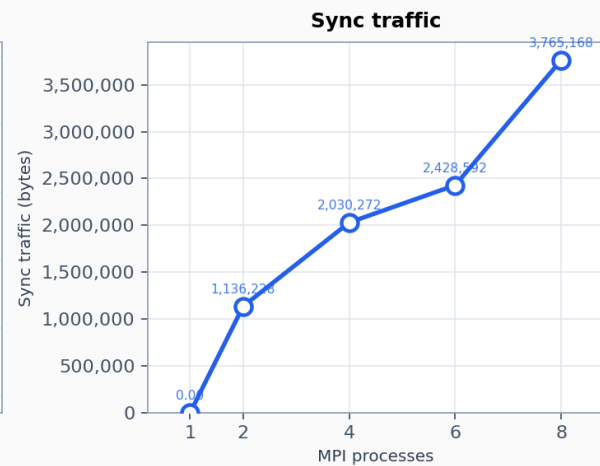
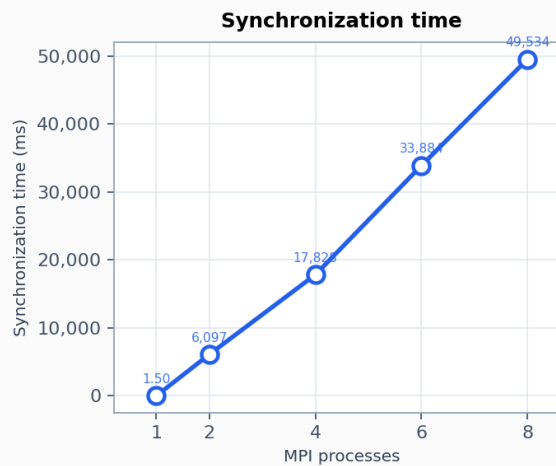
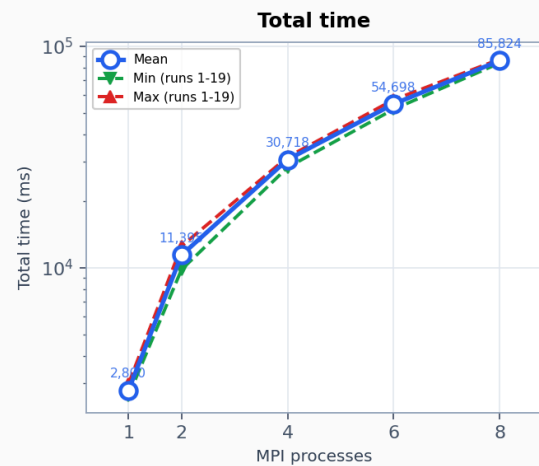
Метрика	Смысл
<code>total_time_exec</code>	среднее время одного run
<code>sync_time</code>	время синхронизаций
<code>sync_bytes</code>	MPI-трафик в фазе sync
<code>graph_construct_time</code>	построение распределённого графа
<code>replication_factor</code>	среднее число копий вершины
<code>replication_edges</code>	среднее число копий вершины рёбер
<code>load_bytes</code> , <code>peak_load_bytes</code>	загрузка проxy-рёбер

BFS

BFS: результаты

BFS scalability — Road network — Pennsylvania

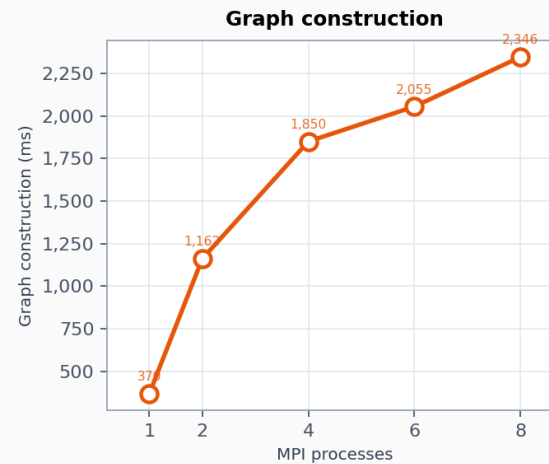
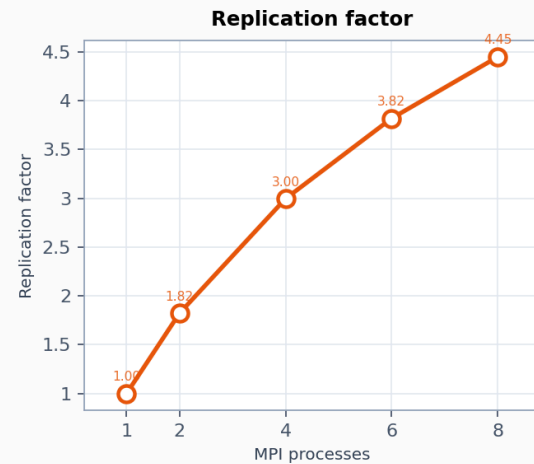
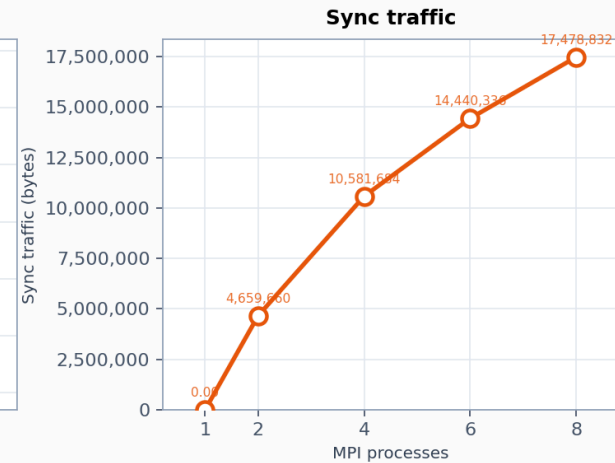
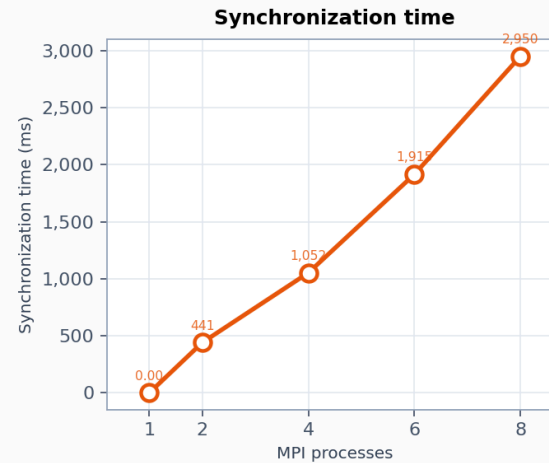
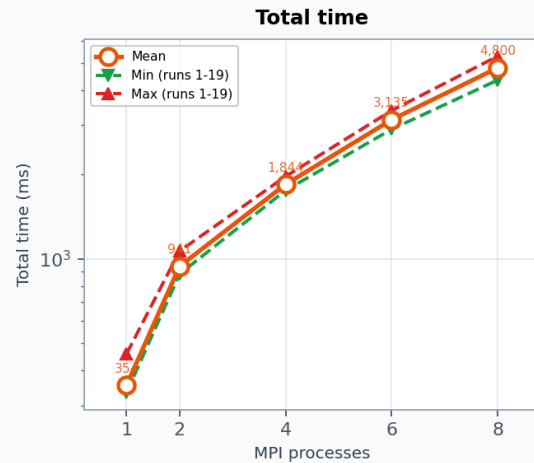
MPI processes: 1 · 2 · 4 · 6 · 8 | threads=2, runs=20



BFS: web-Google

BFS scalability — Web graph — Google

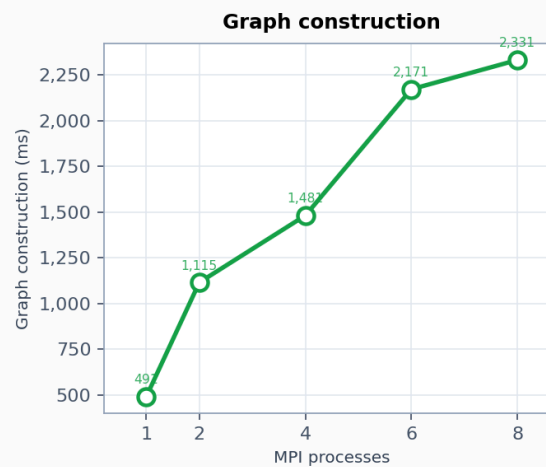
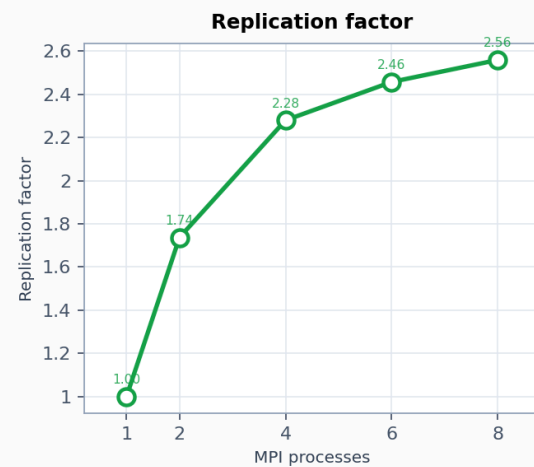
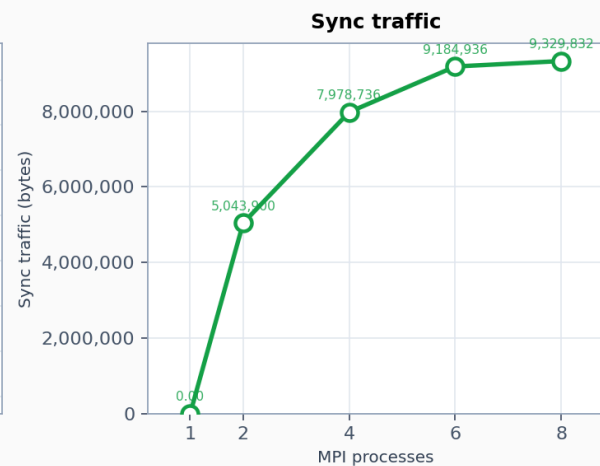
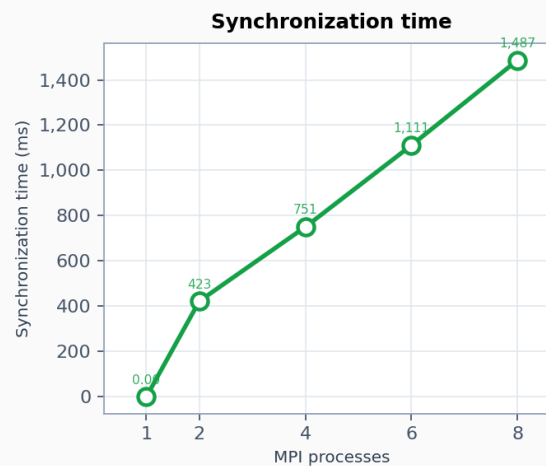
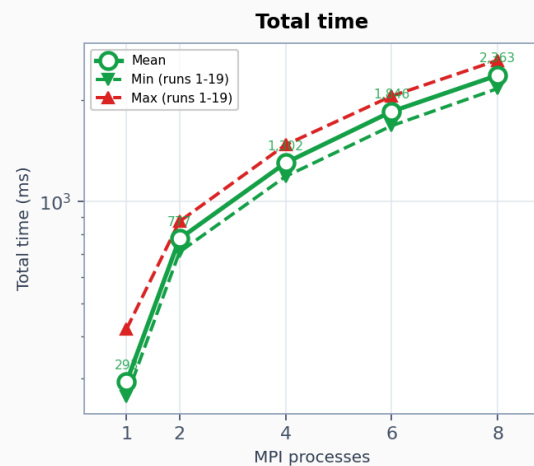
MPI processes: 1 · 2 · 4 · 6 · 8 | threads=2, runs=20



BFS: wiki-talk-temporal

BFS scalability — Wikipedia talk (temporal)

MPI processes: 1 · 2 · 4 · 6 · 8 | threads=2, runs=20



BFS: почему roadNet сильно деградирует

Для roadNet-PA:

- `total_time_exec` : 2800 ms -> 85824 ms
- `sync_time` : 1.5 ms -> 49 534 ms
- количество BFS-раундов: около 317

Причина:

`total sync cost` $\approx$ число BFS-уровней * стоимость sync одного уровня

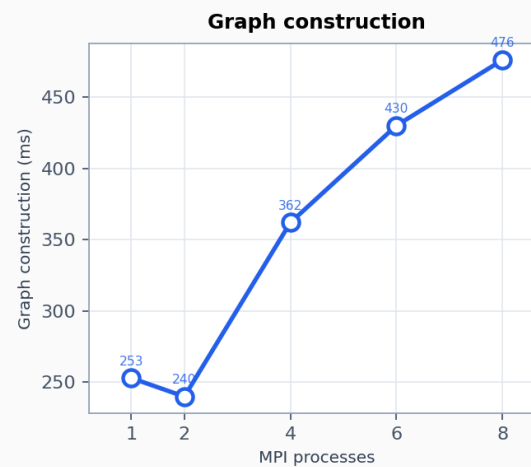
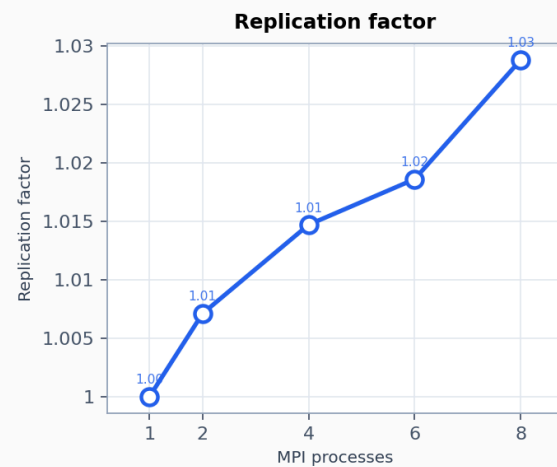
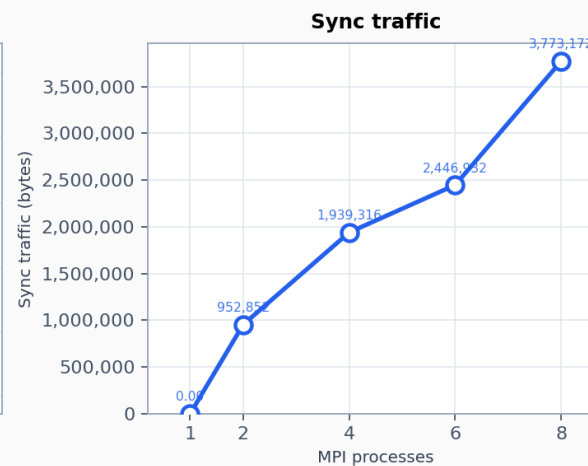
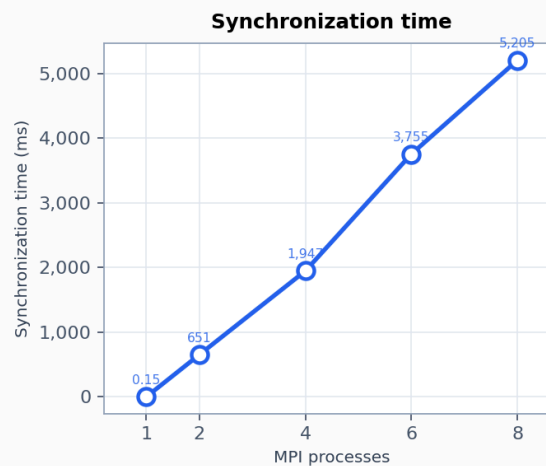
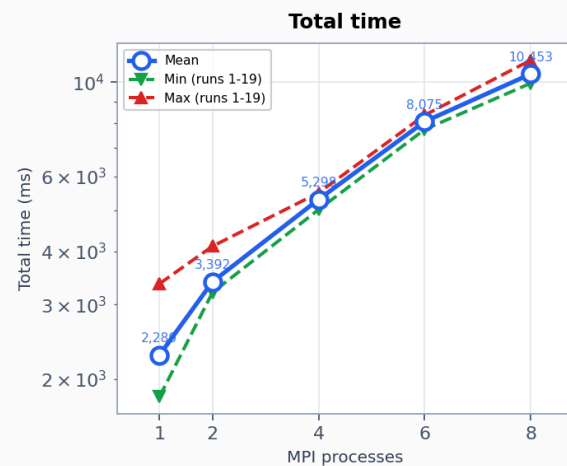
Дорожный граф имеет большой диаметр, поэтому даже маленькая стоимость одной синхронизации умножается на сотни раундов.

PageRank

PageRank: roadNet-PA

PageRank scalability — Road network — Pennsylvania

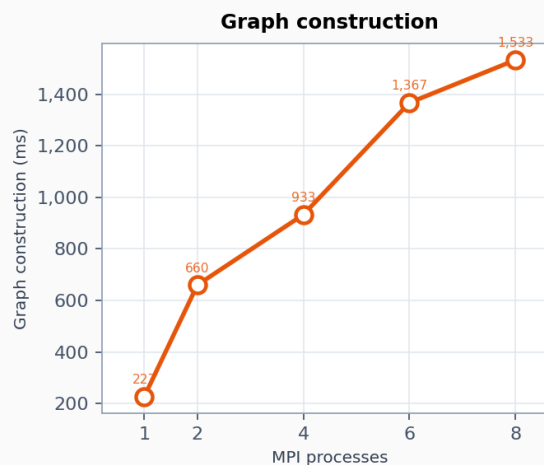
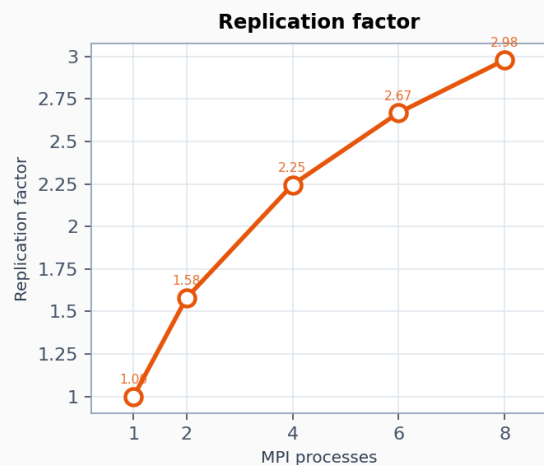
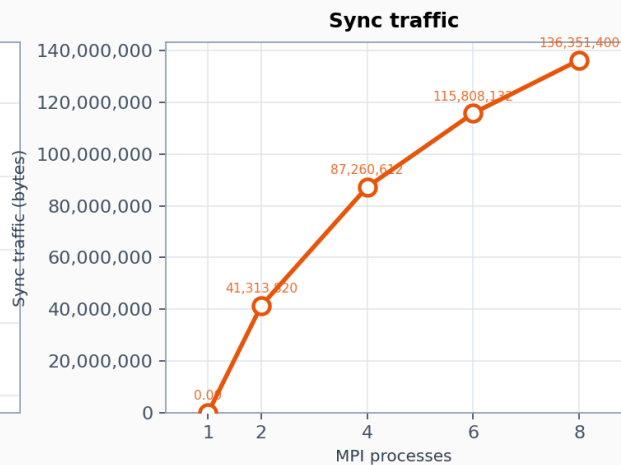
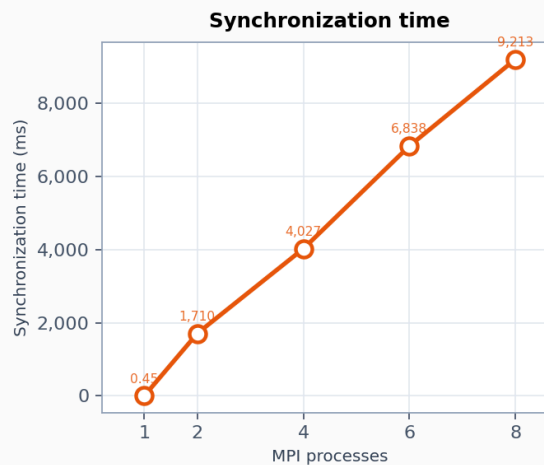
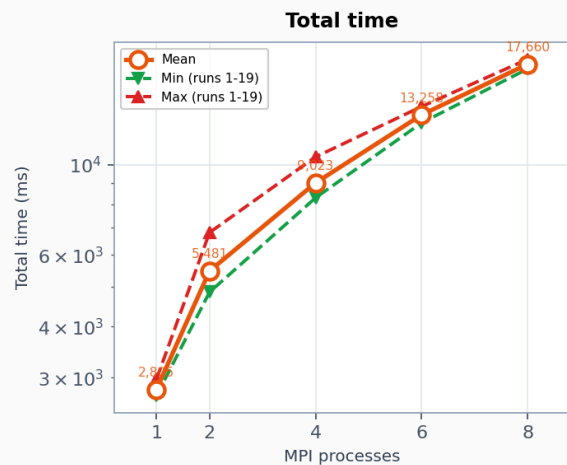
MPI processes: 1 · 2 · 4 · 6 · 8 | threads=2, runs=20



PageRank: web-Google

PageRank scalability — Web graph — Google

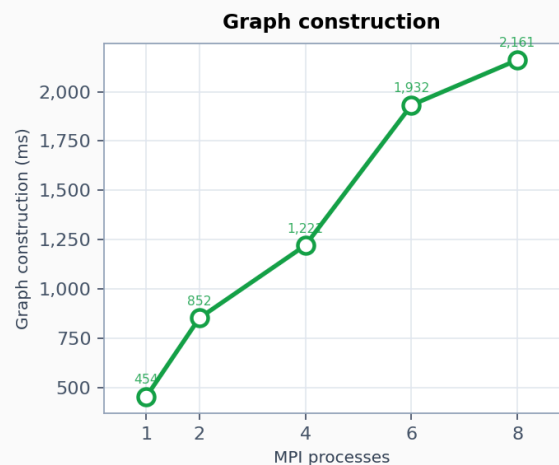
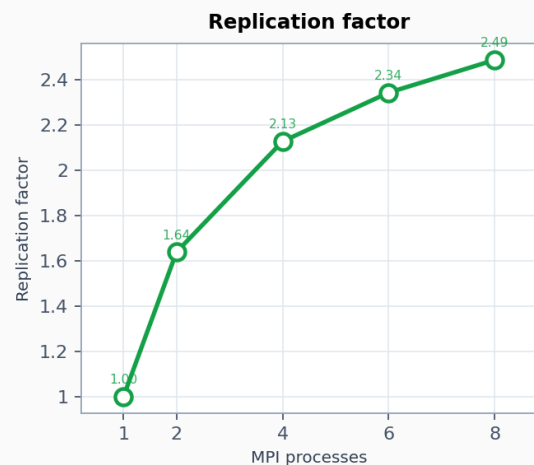
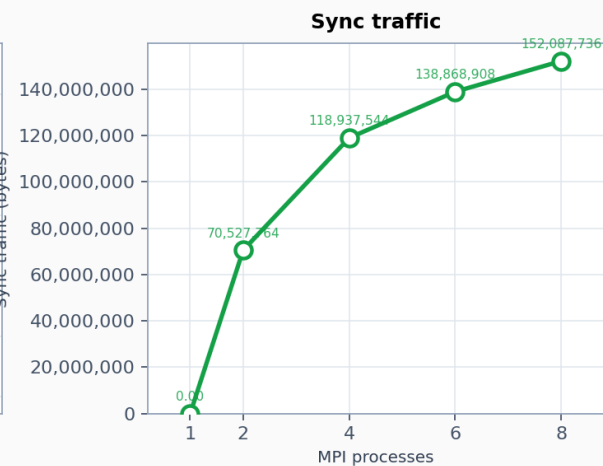
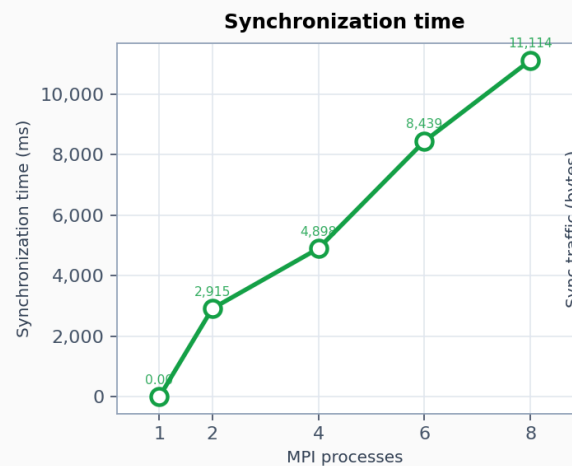
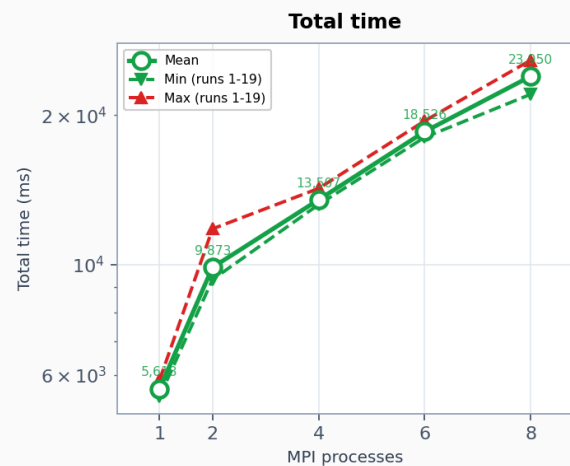
MPI processes: 1 · 2 · 4 · 6 · 8 | threads=2, runs=20



PageRank: wiki-talk-temporal

PageRank scalability — Wikipedia talk (temporal)

MPI processes: 1 · 2 · 4 · 6 · 8 | threads=2, runs=20



PageRank: почему графы деградируют

Для roadNet-PA:

- `replication_factor` : 1.0 -> 1.03
- `sync_bytes` : 0 -> 3.8 MB
- `total_time_exec` : 2280 ms -> 10453 ms

Для web-Google:

- `replication_factor` : 1.0 -> 2.98
- `sync_bytes` : 0 -> 136 MB
- `total_time_exec` : 2805 ms -> 17660 ms

Для wiki-talk:

- `replication_factor` : 1.0 -> 2.49
- `sync_bytes` : 0 -> 152 MB
- `total_time_exec` : 5618 ms -> 23950 ms

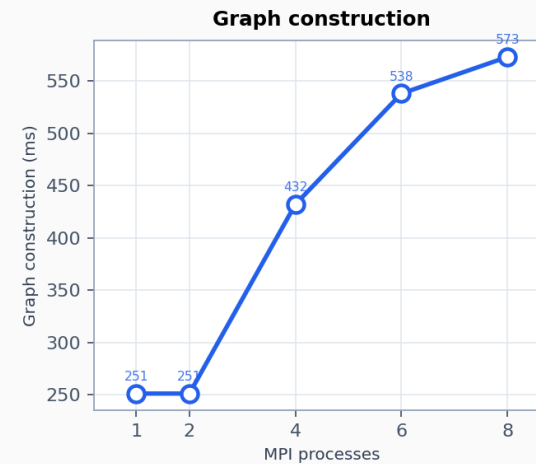
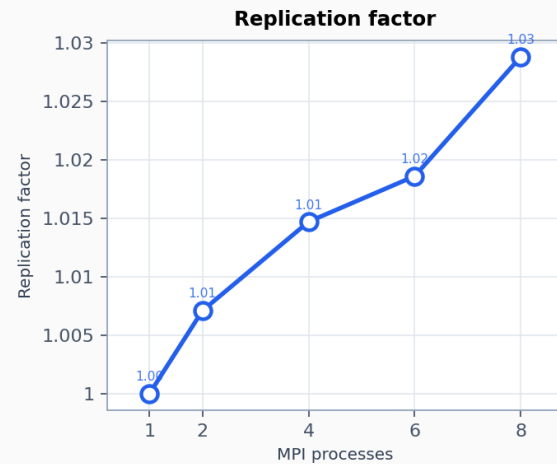
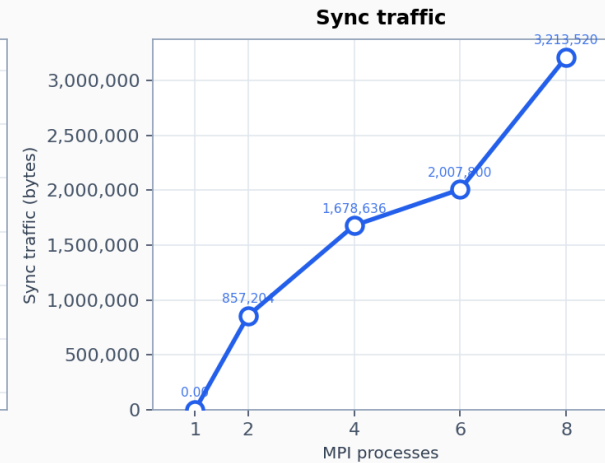
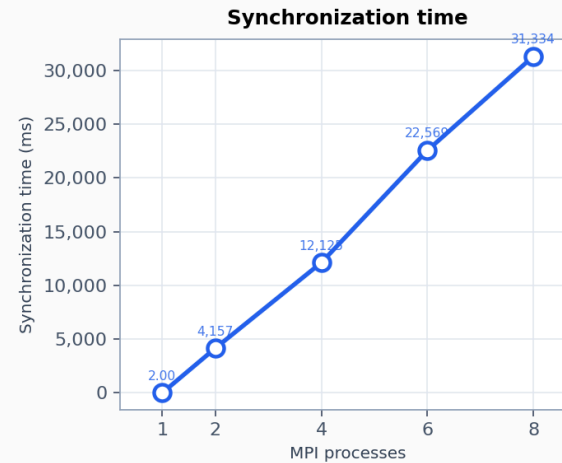
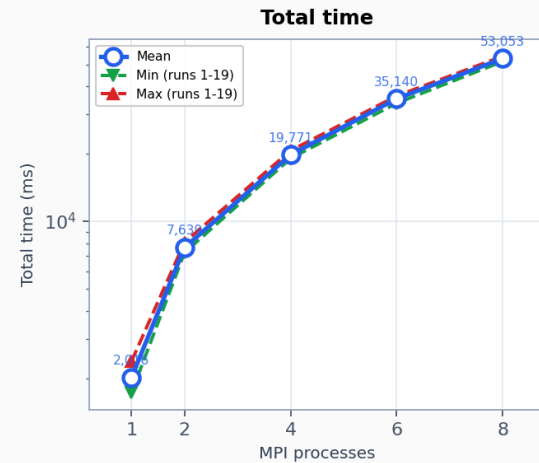
PageRank передаёт значения рангов на границах partition. Поэтому рост числа MPI-процессов увеличивает не только параллелизм, но и объём коммуникации.

SSSP

SSSP: roadNet-PA

SSSP scalability — Road network — Pennsylvania

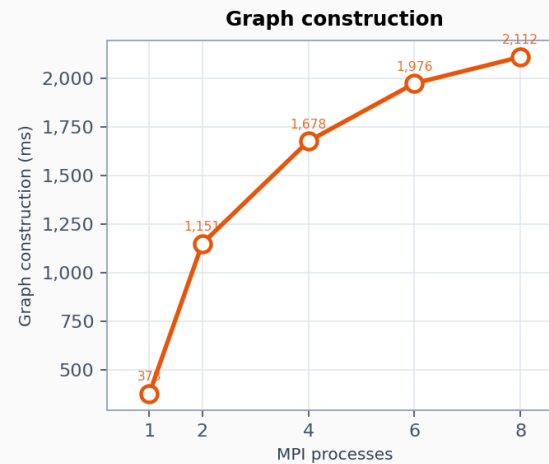
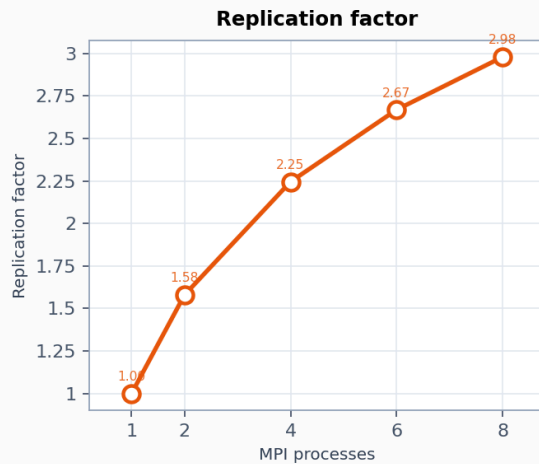
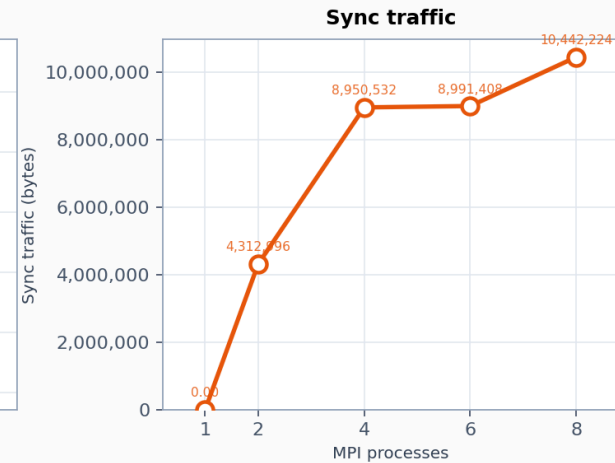
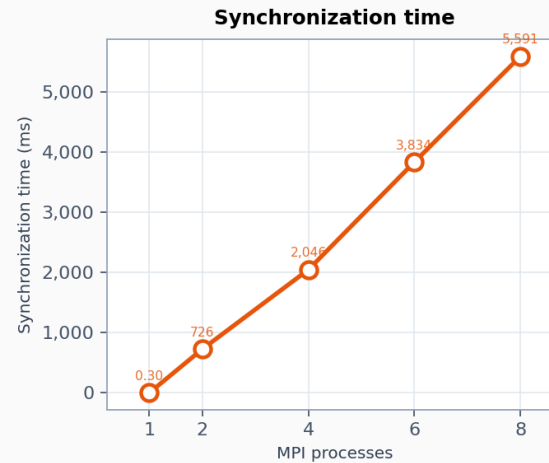
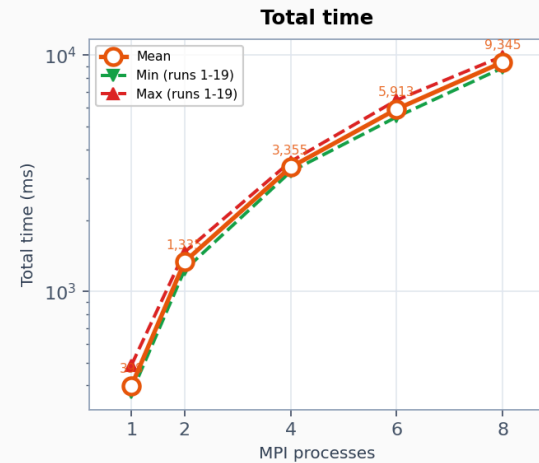
MPI processes: 1 · 2 · 4 · 6 · 8 | threads=2, runs=20



SSSP: web-Google

SSSP scalability — Web graph — Google

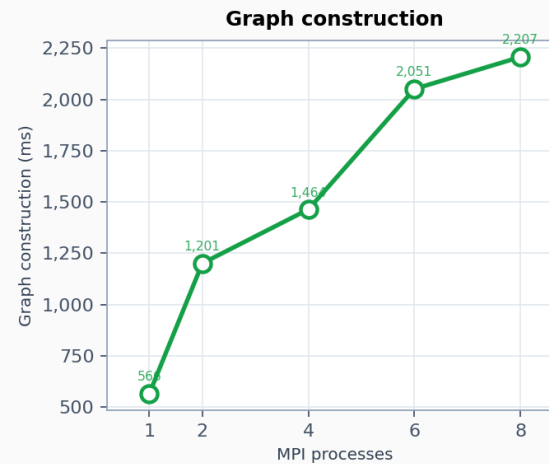
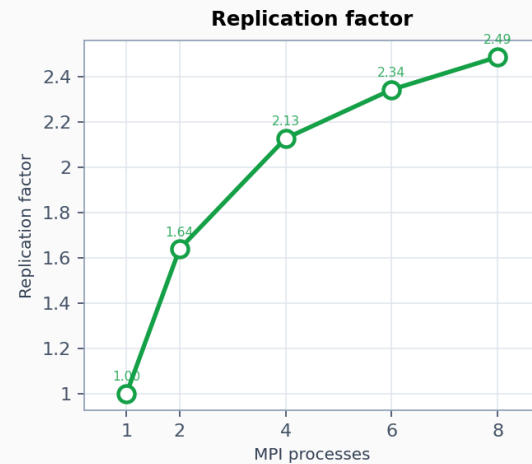
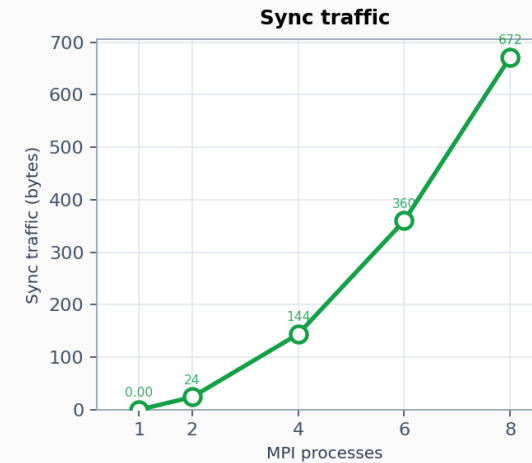
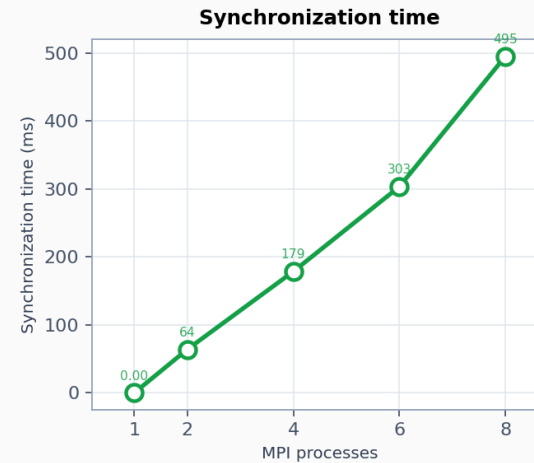
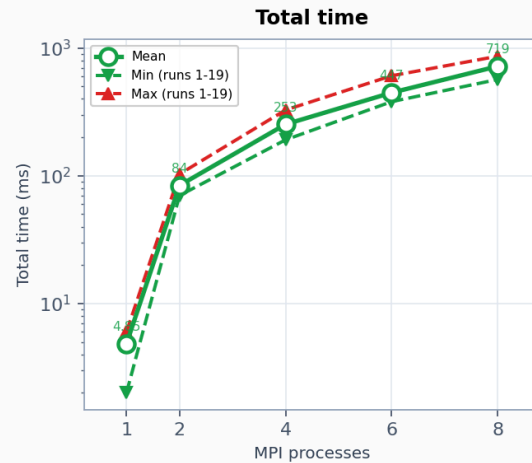
MPI processes: 1 · 2 · 4 · 6 · 8 | threads=2, runs=20



SSSP: wiki-talk-temporal

SSSP scalability — Wikipedia talk (temporal)

MPI processes: 1 · 2 · 4 · 6 · 8 | threads=2, runs=20



SSSP: почему `total_time_exec` деградирует

- `roadNet-PA` : большой диаметр и лимит `maxIterations=200` , поэтому стоимость синхронизаций накапливается: `sync_time` 2 ms -> 31 334 ms.
- `web-Google` : из-за вершин-хабов граф плохо делится на независимые части. Одни и те же вершины приходится хранить на нескольких MPI-процессах, поэтому растёт обмен данными: `sync_bytes` 0 -> 10.4 MB.
- `wiki-talk` : сам запуск очень короткий, поэтому даже 495 ms `sync_time` при MPI=8 полностью доминируют над `total_time_exec` .

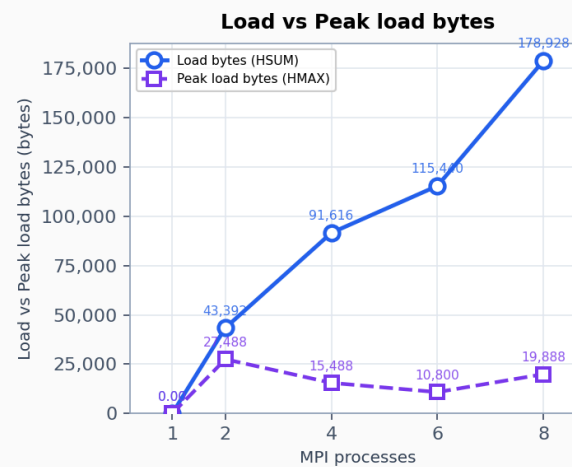
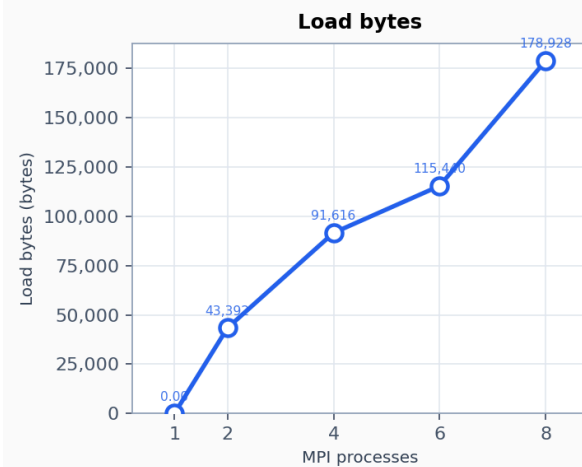
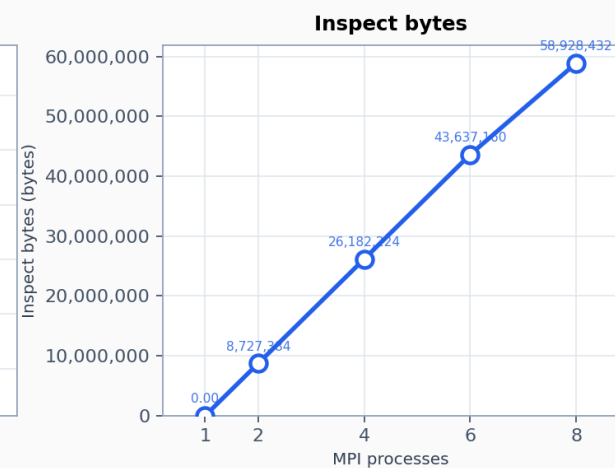
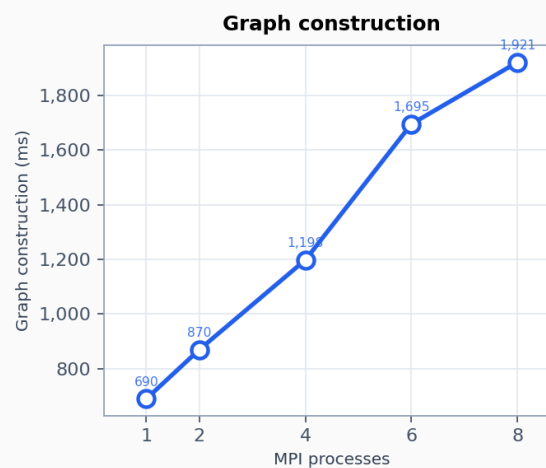
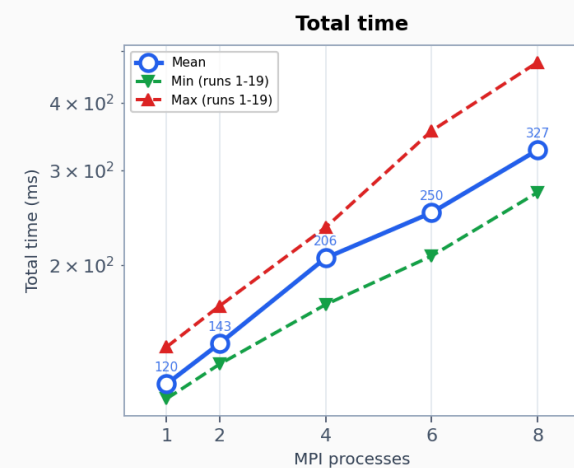
Итог: во всех случаях MPI добавляет синхронизации, проху-данные и ожидание rank'ов быстрее, чем сокращает полное время одного run.

Triangle Counting

Triangle Counting: roadNet-PA

Triangle Counting scalability — Road network — Pennsylvania

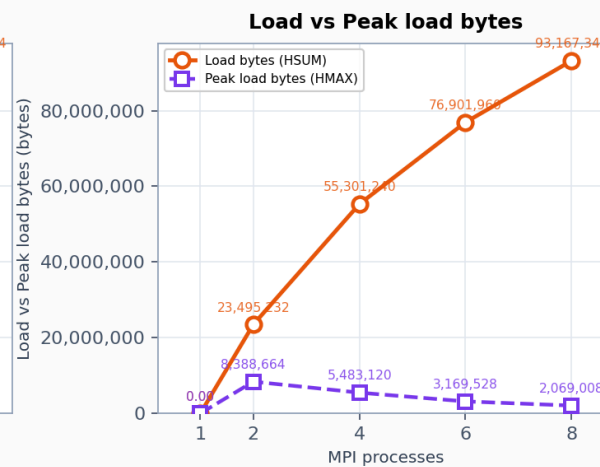
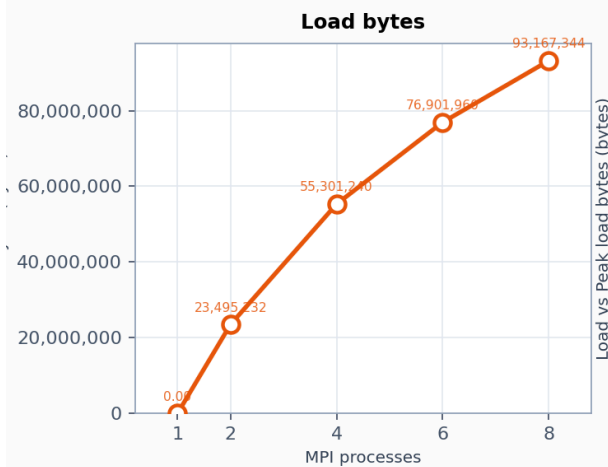
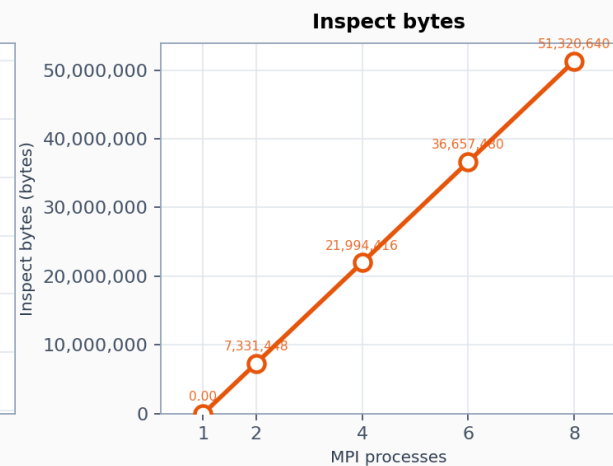
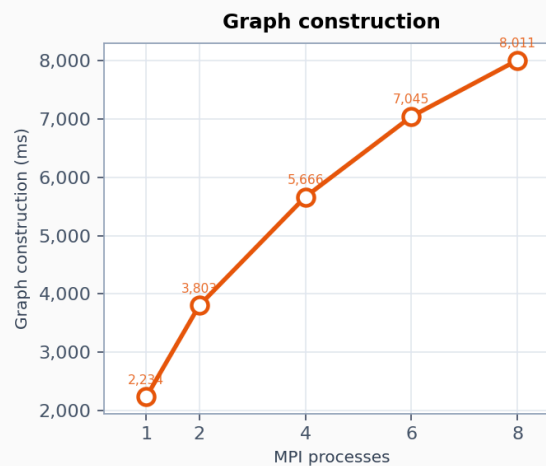
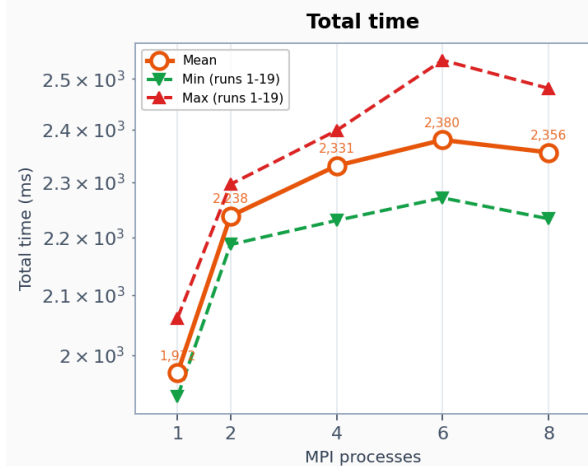
MPI processes: 1 · 2 · 4 · 6 · 8 | threads=2, runs=20



Triangle Counting: web-Google

Triangle Counting scalability — Web graph — Google

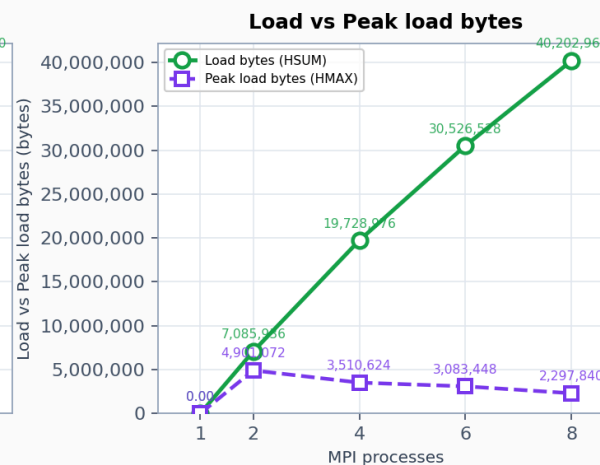
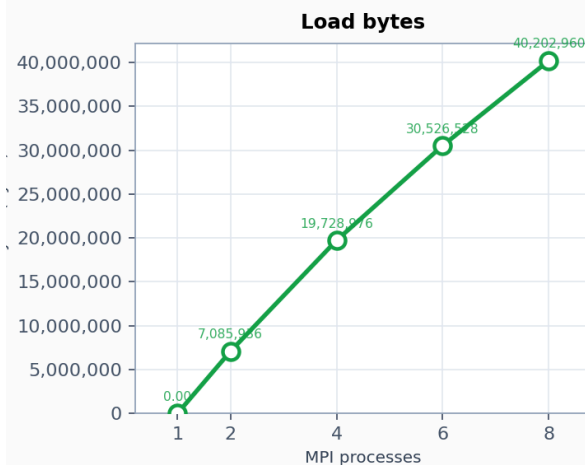
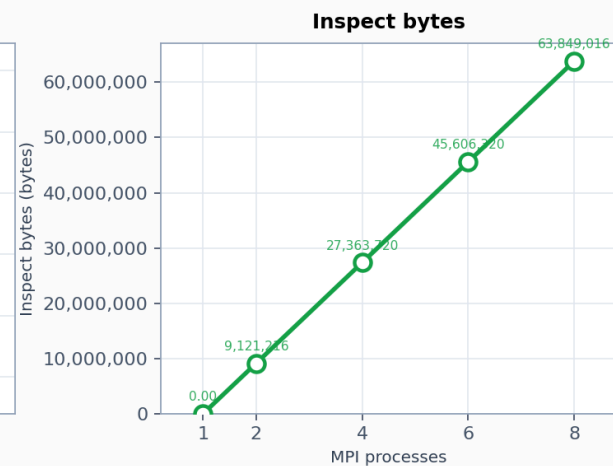
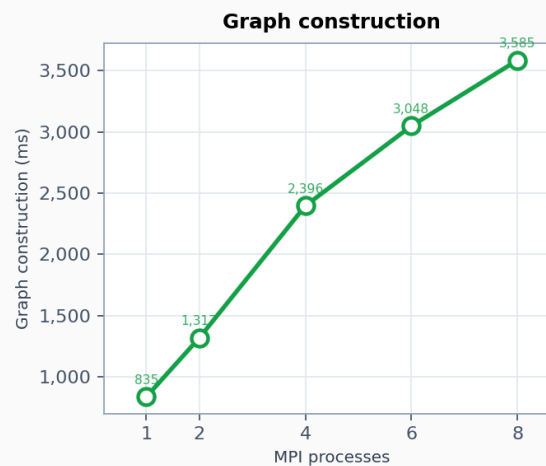
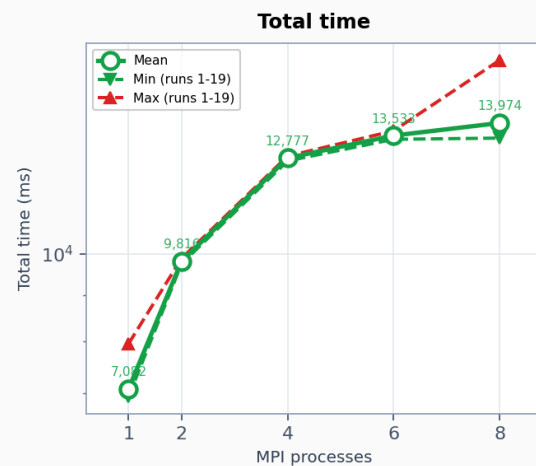
MPI processes: 1 · 2 · 4 · 6 · 8 | threads=2, runs=20



Triangle Counting: wiki-talk-temporal

Triangle Counting scalability — Wikipedia talk (temporal)

MPI processes: 1 · 2 · 4 · 6 · 8 | threads=2, runs=20



ТС: отличие от BFS/PR/SSSP

Для Triangle Counting:

- `sync_time = 0`
- `sync_bytes = 0`
- основной overhead — построение mining-графа, proxy edges и global reduce`

ТС: ключевые наблюдения

Граф	MPI=1	MPI=8	Главная причина
roadNet-PA	120 ms	327 ms	граф простой для ТС: треугольников мало, поэтому overhead MPI становится заметнее самой работы
web-Google	1972 ms	2356 ms	из-за страниц-хабов появляются копии рёбер и больше служебных данных между процессами
wiki-talk	7082 ms	13974 ms	нагрузка делится неравномерно: часть процессов получает более тяжёлые участки графа

Во всех трёх случаях лучший `total_time_exec` — MPI=1.

Сравнение алгоритмов: лучший `total_time_exec`

Алгоритм	roadNet-PA	web-Google	wiki-talk
BFS	MPI=1	MPI=1	MPI=1
PageRank	MPI=1	MPI=1	MPI=1
SSSP	MPI=1	MPI=1	MPI=1
TC	MPI=1	MPI=1	MPI=1

На данной платформе распределённое выполнение увеличивает накладные расходы быстрее, чем сокращает полное время запуска.

Сравнение алгоритмов: деградация MPI=8

Алгоритм / граф	MPI=1	MPI=8	Замедление
BFS roadNet-PA	2800 ms	85824 ms	30.6x
PageRank web-Google	2805 ms	17660 ms	6.3x
SSSP web-Google	398 ms	9345 ms	23.5x
TC web-Google	1972 ms	2356 ms	1.2x

Главный вывод: при выбранной конфигурации MPI=8 не даёт лучшего `total_time_exec` ни для одного алгоритма и графа.

Общий вывод

Лучший `total_time_exec` для всех алгоритмов и графов даёт `MPI=1`.

Почему MPI не ускорил запуск:

- `roadNet-PA` : длинные пути -> много синхронизаций.
- `web-Google` : страницы-хабы -> копии вершин и больше обмена.
- `wiki-talk` : неравная нагрузка -> ожидание самого медленного процесса.
- TC: overhead построения mining-графа и проху-рёбер.

Итог: на одном хосте накладные расходы MPI оказались больше выигрыша от распределения работы.